

Lab D

Shift Register



课程名称：数字逻辑电路设计

作者：王传宇 3250102681

专业：计算机科学与技术

邮箱：3250102681@zju.edu.cn

时间：2026-6-4

地点：紫金港东四 509 室

第一部分：操作方法与实验步骤

1.1 移位寄存器 ShiftReg8b 的实现与仿真

根据实验要求，使用 Verilog 编写 8 位右移移位寄存器模块 ShiftReg8b，其功能为：

- shiftn_loadp = 1 时，在时钟上升沿并行加载 par_in；
- shiftn_loadp = 0 时，在时钟上升沿右移一位，最高位移入 shift_in。

核心代码片段如下：

```
always @(posedge clk) begin
    if (shiftn_loadp)
        shift_reg <= par_in;
    else
        shift_reg <= {shift_in, shift_reg[7:1]};
end
```

编写测试平台 tb_ShiftReg8b.v，施加并行加载与多次移位激励，在 Vivado Simulator 中运行仿真，得到波形。

1.2 跑马灯系统顶层设计

建立 Vivado 工程 Marquee，加入以下模块：

- clk_1s.v (1 Hz 分频)
- CreateNumber.v (按键自增寄存器)
- DisplayNumber.v (七段码动态扫描显示)
- ShiftReg8b.v (8 位移位寄存器)
- Marquee_top.v (顶层模块)

顶层模块 Marquee_top 的关键连接逻辑：

```
// regA 和 regB 取自 CreateNumber 输出的 num
assign regA = num[15:12];
assign regB = num[11:8];

// CreateNumber 按键映射：SW[1]控制regA，SW[0]控制regB
CreateNumber u_create (
    .btn({2'b00, SW[0], SW[1]}),
    .num(num)
);
```

```
// 移位寄存器并行输入 {regA, regB}
ShiftReg8b u_shift (
    .clk(clk_1s),
    .shiftn_loadp(SW[2]),
    .shift_in(SW[3]),
    .par_in({regA, regB}),
    .Q(LED)
);

// 数码管显示 regA、regB, 后两位置零
DisplayNumber u_display (
    .clk(clk),
    .rst(1'b0),
    .hexs({regA, regB, 8'h00}),
    .points(4'b0000),
    .LEs(4'b0000),
    .AN(AN),
    .SEGMENT(SEGMENT)
);
```

添加约束文件，为时钟、拨码开关、LED、数码管分配正确引脚。综合、实现、生成比特流后下载到开发板。

第二部分：实验结果与分析

2.1 移位寄存器仿真



Figure 1: ShiftReg8b 仿真波形

波形分析：

- 初始时刻 $\text{shiftn_loadp}=1$ ，第一个时钟上升沿将 $\text{par_in}=10110011$ 加载到 Q。
- 随后 $\text{shiftn_loadp}=0$ ， shift_in 依次输入 1、0、1，每个时钟周期 Q 右移一位，最高位被 shift_in 取代，最低位丢弃，输出序列符合右移逻辑。
- 再次拉高 shiftn_loadp ，加载新值 00001111，验证并行加载可随时打断移位。
- 再次移位两次，移入 $\text{shift_in}=0$ ，波形正确。

2.2 寄存器自增与 LED 显示

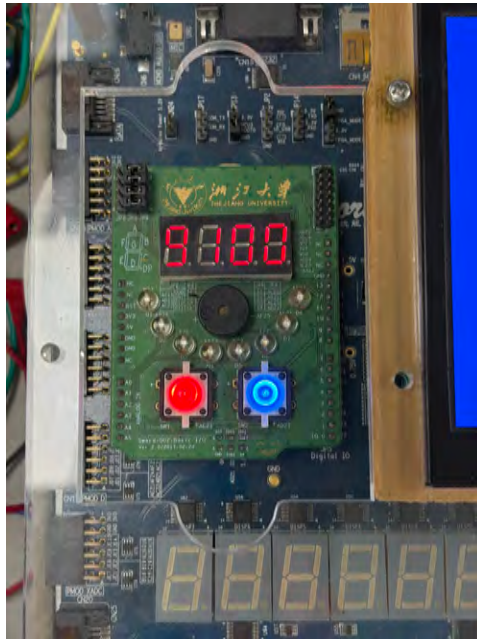


Figure 2: SW[1:0] 自增至 9 和 1，数码管显示 9100

通过 SW[1] 和 SW[0] 的上升沿分别使 regA 和 regB 自增，数码管左起第一位显示 9，第二位显示 1，后两位为 0。验证了 CreateNumber 模块的功能和数码管显示的正确性。



Figure 3: SW[2]=1 时 LED 显示 {regA, regB}=9 和 1

当 SW[2]=1 时，移位寄存器并行加载 {9,1}，8 个 LED 的亮灭对应 1001 0001，直观显示出两个 4 位寄存器的二进制值。

2.3 跑马灯效果

将 $SW[2]=0$ ，移位寄存器进入串行移位模式，通过 $SW[3]$ 控制移入的位。下面一组照片记录了两次完整的 8 位移位过程：前四张为 $SW[3]=1$ ，后四张为 $SW[3]=0$ 。

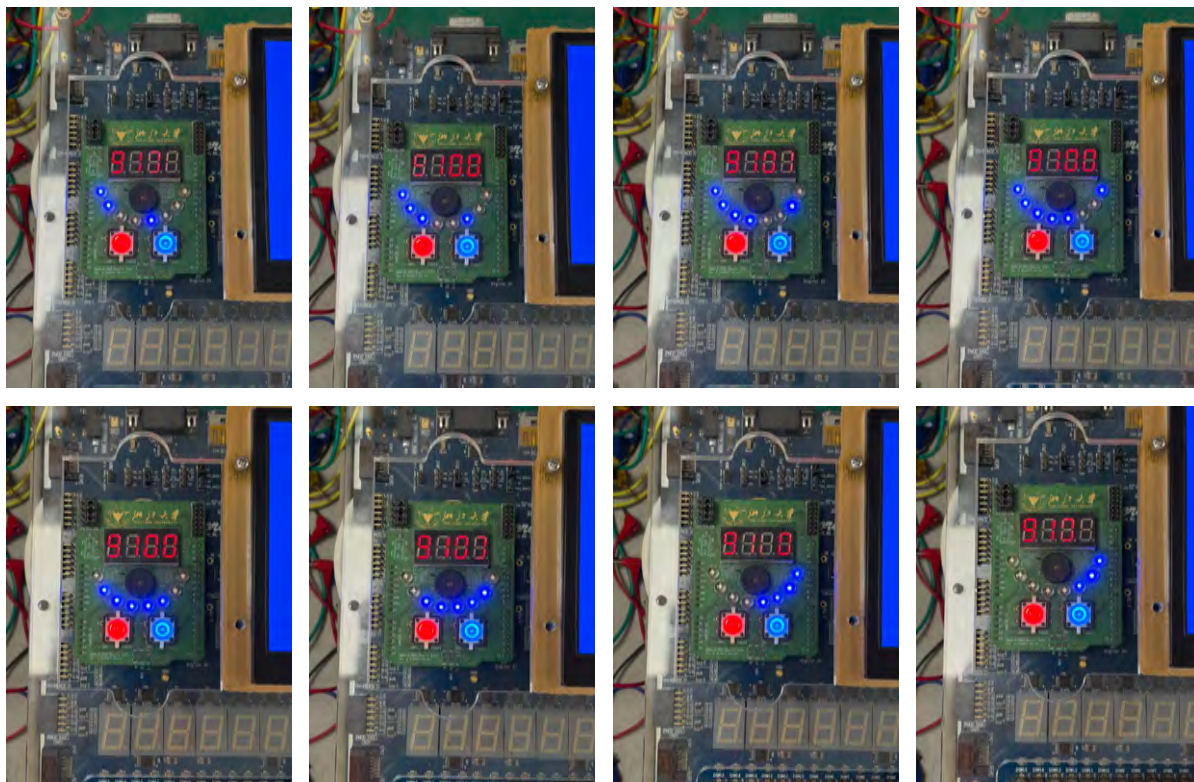


Figure 4: 跑马灯移位过程：上排 $SW[3]=1$ 右移，下排 $SW[3]=0$ 右移
初始加载值 01010001，随着时钟节拍，LED 向右流动。移入 1 时，左侧不断补入亮点；移入 0 时，亮点逐渐移出，整体灯效清晰流畅，验证了移位寄存器在 1 Hz 时钟驱动下的跑马灯功能。

第三部分：讨论与心得

本次实验在实现跑马灯的过程中，遇到了一个典型的端口映射错误。最初顶层模块中 `CreateNumber` 的按钮拼接为 `.btn({SW[1], 1'b0, SW[0], 1'b0})`，导致 `SW[1]` 和 `SW[0]` 实际控制的是不参与显示的寄存器低位，而 `regA` 和 `regB` 对应的控制端被接地，因此拨动开关时数码管上数值完全不变。更换多块开发板仍无法解决，最终通过仔细阅读 `CreateNumber` 的源代码，发现其内部 `btn[0]` 和 `btn[1]` 分别控制 `num[15:12]` 和 `num[11:8]`，这才意识到是信号映射错误。修正为 `.btn({2'b00, SW[0], SW[1]})` 后功能立即正常。

修正后又遇到 Vivado 布局布线报错 `CLOCK_DEDICATED_ROUTE`，原因是 `SW[1]` 和 `SW[0]` 作为 `posedge` 触发信号被工具识别为时钟，而所分配的引脚不是专用时钟引脚。通过添加约束 `set_property CLOCK_DEDICATED_ROUTE FALSE` 顺利解决。

通过本实验，我深刻体会到模块接口的仔细匹配至关重要，尤其是多位宽控制信号的拼接顺序。同时学会了如何处理非时钟引脚的边沿触发约束。整个设计从底层移位寄存器到顶层系统集成的流程得到了完整锻炼，对同步电路、开关消抖、动态显示等知识有了更实际的理解。

附录

4.1 源代码清单

4.1.1. ShiftReg8b.v

```
module ShiftReg8b(  
    input      clk,  
    input      shiftn_loadp,    // 0: shift right, 1: parallel load  
    input      shift_in,        // serial data input for shifting  
    input [7:0] par_in,         // 8-bit parallel data input  
    output [7:0] Q               // current state output  
);  
  
    reg [7:0] shift_reg;        // internal 8-bit register  
  
    // Synchronous operation on rising edge of clk  
    always @(posedge clk) begin  
        if (shiftn_loadp)      // parallel load mode  
            shift_reg <= par_in;  
        else                    // shift right mode  
            shift_reg <= {shift_in, shift_reg[7:1]};  
        end  
  
    assign Q = shift_reg;       // output the current state  
  
endmodule
```

4.1.2. tb_ShiftReg8b.v

```
`timescale 1ns / 1ps  
  
module tb_ShiftReg8b;  
  
    reg clk;  
    reg shiftn_loadp;  
    reg shift_in;  
    reg [7:0] par_in;  
    wire [7:0] Q;  
  
    // Instantiate the Unit Under Test (UUT)  
    ShiftReg8b uut (  
        .clk(clk),
```



```

        .shiftn_loadp(shiftn_loadp),
        .shift_in(shift_in),
        .par_in(par_in),
        .Q(Q)
    );

// Clock generation: 10 ns period
always #5 clk = ~clk;

initial begin
    // Initialize signals
    clk = 0;
    shiftn_loadp = 1;      // Start with load mode to set an initial value
    shift_in = 0;
    par_in = 8'b10110011;  // arbitrary initial pattern

    // Parallel load
    #10;                  // at first rising edge, load par_in (10110011)
    $display("After load: Q = %b", Q);

    // Switch to shift mode and shift in a '1'
    shiftn_loadp = 0;
    shift_in = 1;
    #10;                  // shift right, LSB lost, MSB becomes 1 -> Q =
1_1011001
    $display("Shift 1: Q = %b", Q);

    // Shift in a '0'
    shift_in = 0;
    #10;                  // Q = 0_1101100
    $display("Shift 2: Q = %b", Q);

    // Shift in a '1'
    shift_in = 1;
    #10;                  // Q = 1_0110110
    $display("Shift 3: Q = %b", Q);

    // Load a new value
    shiftn_loadp = 1;
    par_in = 8'b00001111;
    #10;                  // Q should become 00001111
    $display("After reload: Q = %b", Q);

    // Switch back to shift, shift in '0' twice

```

```

    shiftn_loadp = 0;
    shift_in = 0;
    #10; // Q = 0_0000111
    $display("Shift 4: Q = %b", Q);
    #10; // Q = 0_0000011
    $display("Shift 5: Q = %b", Q);

    $finish;
end

// Monitor changes
always @(posedge clk) begin
    #1; // small delay to capture stable outputs
    $display("Time=%t: Q = %b", $time, Q);
end

endmodule

```

4.1.3. Marquee_top.v

```

module Marquee_top(
    input      clk,           // 100 MHz main clock
    input  [3:0] SW,          // SW[1:0] for inc, SW[2]=load/shift,
    SW[3]=shift_in
    output [7:0] LED,         // shift register output Q[7:0]
    output [3:0] AN,
    output [7:0] SEGMENT
);

    wire [15:0] hexs_display;
    wire [15:0] num;
    wire [3:0]  regA, regB;
    wire clk_1s;

    // Extract regA and regB from CreateNumber (choose the appropriate nibbles)
    assign regA = num[15:12]; // controlled by SW[1] (btn[3])
    assign regB = num[11:8];  // controlled by SW[0] (btn[2])

    // Display regA on leftmost digit, regB on second left, others blank (show
    0)
    assign hexs_display = {regA, regB, 8'b0000_0000};

    // 1 Hz clock for shift register (slow visual effect)
    clk_1s clk_div (
        .clk(clk),

```

```

        .clk_1s(clk_1s)
    );

    // Button-triggered increment module
    CreateNumber u_create (
        .btn({2'b00, SW[0], SW[1]}), // btn[1]=SW[0] (regB), btn[0]=SW[1]
        (regA)
        .num(num)
    );

    // 7-segment display
    DisplayNumber u_display (
        .clk(clk),
        .rst(1'b0),
        .hexs(hexs_display),
        .points(4'b0000),
        .LEs(4'b0000),
        .AN(AN),
        .SEGMENT(SEGMENT)
    );

    // 8-bit right-shift register (controlled by SW[2] and SW[3])
    ShiftReg8b u_shift (
        .clk(clk_1s),
        .shiftn_loadp(SW[2]), // 1: load {regA, regB}, 0: shift right
        .shift_in(SW[3]),
        .par_in({regA, regB}),
        .Q(LED)
    );

endmodule

```

4.1.4. clk_1s.v

```

// clk_1s.v: 100 MHz -> 1 Hz clock divider
module clk_1s(
    input  clk,
    output reg clk_1s
);

    reg [25:0] cnt; // 50,000,000 -> half period
    always @(posedge clk) begin
        if (cnt == 49_999_999) begin
            cnt <= 0;
            clk_1s <= ~clk_1s;
        end else begin

```

```

        cnt <= cnt + 1;
    end
end
initial begin
    cnt = 0;
    clk_1s = 0;
end
endmodule

```

4.1.5. CreateNumber.v 和 DisplayNumber.v 由之前的实验提供，此处不再重复列出。

4.2 约束文件

```

# Main clock (100 MHz)
set_property PACKAGE_PIN AC18 [get_ports clk]
set_property IOSTANDARD LVCMOS18 [get_ports clk]
create_clock -period 10.000 -name clk [get_ports clk]

set_property CLOCK_DEDICATED_ROUTE FALSE [get_nets {SW_IBUF[0]}]
set_property CLOCK_DEDICATED_ROUTE FALSE [get_nets {SW_IBUF[1]}]

# Slide switches SW[3:0]
set_property PACKAGE_PIN AA10 [get_ports {SW[0]}]
set_property PACKAGE_PIN AB10 [get_ports {SW[1]}]
set_property PACKAGE_PIN AA13 [get_ports {SW[2]}]
set_property PACKAGE_PIN AA12 [get_ports {SW[3]}]
set_property IOSTANDARD LVCMOS15 [get_ports {SW[0]}]
set_property IOSTANDARD LVCMOS15 [get_ports {SW[1]}]
set_property IOSTANDARD LVCMOS15 [get_ports {SW[2]}]
set_property IOSTANDARD LVCMOS15 [get_ports {SW[3]}]

# 8 LEDs (shift register Q[7:0])
# Use Arduino LED pins from previous constraints
set_property PACKAGE_PIN AF24 [get_ports {LED[0]}]
set_property PACKAGE_PIN AE21 [get_ports {LED[1]}]
set_property PACKAGE_PIN Y22 [get_ports {LED[2]}]
set_property PACKAGE_PIN Y23 [get_ports {LED[3]}]
set_property PACKAGE_PIN AA23 [get_ports {LED[4]}]
set_property PACKAGE_PIN Y25 [get_ports {LED[5]}]
set_property PACKAGE_PIN AB26 [get_ports {LED[6]}]
set_property PACKAGE_PIN W23 [get_ports {LED[7]}]
set_property IOSTANDARD LVCMOS33 [get_ports {LED[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {LED[1]}]

```

```
set_property IOSTANDARD LVCMOS33 [get_ports {LED[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {LED[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {LED[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {LED[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {LED[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {LED[7]}]

# 7-segment anode (AN[3:0]) and segments (SEGMENT[7:0])
# Use the same pinout as in previous labs
set_property PACKAGE_PIN AD21 [get_ports {AN[0]}]
set_property PACKAGE_PIN AC21 [get_ports {AN[1]}]
set_property PACKAGE_PIN AB21 [get_ports {AN[2]}]
set_property PACKAGE_PIN AC22 [get_ports {AN[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {AN[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {AN[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {AN[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {AN[3]}]

set_property PACKAGE_PIN AB22 [get_ports {SEGMENT[0]}]
set_property PACKAGE_PIN AD24 [get_ports {SEGMENT[1]}]
set_property PACKAGE_PIN AD23 [get_ports {SEGMENT[2]}]
set_property PACKAGE_PIN Y21 [get_ports {SEGMENT[3]}]
set_property PACKAGE_PIN W20 [get_ports {SEGMENT[4]}]
set_property PACKAGE_PIN AC24 [get_ports {SEGMENT[5]}]
set_property PACKAGE_PIN AC23 [get_ports {SEGMENT[6]}]
set_property PACKAGE_PIN AA22 [get_ports {SEGMENT[7]}]
set_property IOSTANDARD LVCMOS33 [get_ports {SEGMENT[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {SEGMENT[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {SEGMENT[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {SEGMENT[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {SEGMENT[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {SEGMENT[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {SEGMENT[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {SEGMENT[7]}]
```